Phân tích thuật toán

Bài giảng 02

PGS. TS. Nguyễn Thanh Bình

Khoa Toán - Tin học, Trường Đại học Khoa học Tự nhiên
Đại học Quốc gia TP. Hồ Chí Minh

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

Quy nạp toán học

Để chứng minh một tính chất nào đó đúng với mọi số tự nhiên n:

1. Chứng minh tính chất đúng với $n = 1$.
2. Chứng minh với mọi $n \geq 1$, nếu tính chất đúng tại n, thì nó cũng đúng với $n + 1$.

Quy nạp toán học

Strong induction:

1. Chứng minh tính chất đúng với $n = 1$.
2. Chứng minh với mọi $n \geq 1$, nếu tính chất đúng với $1 \leq m \leq n$, thì nó cũng đúng với $n + 1$.

Quy nạp toán học

Ví dụ:

- Với mọi $n \in \mathbb{N}$:

$$1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

.

- Với mọi $n \in \mathbb{N}$, nếu $1 + x > 0$ thì:

$$(1+x)^n \geq 1 + nx$$

Quy nạp toán học

- Chứng minh rằng một cây nhị phân hoàn chỉnh với cấp k có $2^k - 1$ nút.
- Chứng minh với tất cả $n \geq 1$, ta có:

$$\sum_{i=1}^{n} \frac{1}{2^i} < 1$$

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

Tính đúng đắn của thuật toán

Làm sao để chứng minh tính đúng đắn của một thuật toán được đưa ra?

Hai phương pháp phổ biến để kiểm tra tính đúng đắn của thuật toán:

- Testing.
- Correctness proof.

Tính đúng đắn của thuật toán

So sánh hai phương pháp:

- Testing:
  - Kiểm tra thuật toán dựa vào những dữ liệu đưa vào.
  - Khó phát hiện ra toàn bộ lỗi.
- Correctness proof:
  - Chứng minh chặt chẽ bằng toán học.
  - Có thể có lỗi.

$\Longrightarrow$ Nên kết hợp cả hai phương pháp.

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

Thuật toán đệ quy

- Để chứng minh tính đúng đắn của một thuật toán hồi quy $P(n)$, chúng ta thường dùng phương pháp quy nạp toán học dựa vào kích thước của bài toán.
- Cần chứng minh thuật toán hồi quy chỉ lặp trong hữu hạn lần.
- Bước quy nạp: giả sử thuật toán cho kết quả đúng với mọi $k \leq n$, chứng minh $P(n+1)$ cho kết quả đúng.

Thuật toán Đệ quy số Fibonacci

Số Fibonacci:

$$F(n+2) = F(n+1) + F(n)$$
$$F(0) = 1, F(1) = 1$$

Cho thuật toán *fibo*(n) như sau:

```
Function fibo(n)
{
    if n ≤ 1, return 1.
    else return fibo(n-1) + fibo(n-2)
}
```

Thuật toán Đệ quy số Fibonacci

- Chứng minh rằng: $fibo(n) = F(n)$ với mọi $n \in \mathbb{N}$.
- Giả sử: $fibo(k) = F(k), k \leq n$
- Cần chứng minh: $fibo(n+1) = F(n+1)$

Recursive Maximum

Cho mảng $A[1...n]$. Tìm phần tử lớn nhất của mảng.
Thuật toán:

```
function maximum(n)
{
    if n ≤ 1, return A(1)
    else return max(maximum(n-1), A(n));
}
```

Chứng minh rằng:

$$\texttt{maximum(n) = max(A[1...n])}$$

Recursive Multiplication

Thuật toán *Recursive Multiplication*:

```
function multiply(x, y)
{
    if (y = 0) return 0;
    if y%2 != 0:
        return multiply(2x, [y/2]) + x;
    else return multiply(2x, [y/2]);
}
```

Chứng minh rằng:

$$\texttt{multiply(x, y)} = xy$$

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

Thuật toán không đệ quy

Để chứng minh tính đúng đắn của một thuật toán không hồi quy, chúng ta thường tiến hành một số bước:

- Phân tích thuật toán trong từng vòng lặp (nếu có).
- Kiểm tra tính dừng của từng vòng lặp.
- Chứng minh tính chính xác của thuật toán.

Iterative Fibonacci numbers

Thuật toán:

```
function fibo(n)
{
    if n = 0, return 1
    else
        a = 1; b = 1; i = 2;
        while (i ≤ n) do
            c = a + b, a = b, b = c, i = i + 1;
        return b;
}
```

Chứng minh rằng:

$$\texttt{fibo(n)} = \texttt{F(n)}$$

Iterative Maximum

Thuật toán:

```
function maximum(A, n)
{
    m = A[1];
    i = 2;
    while (i ≤ n) do
        if A[i] > m then m = A(i);
        i = i + 1;
    return m
}
```

Chứng minh rằng:

$$\texttt{maximum(A, n)} = \max \texttt{A[1...n]}$$

Iterative Multiplication

Thuật toán:

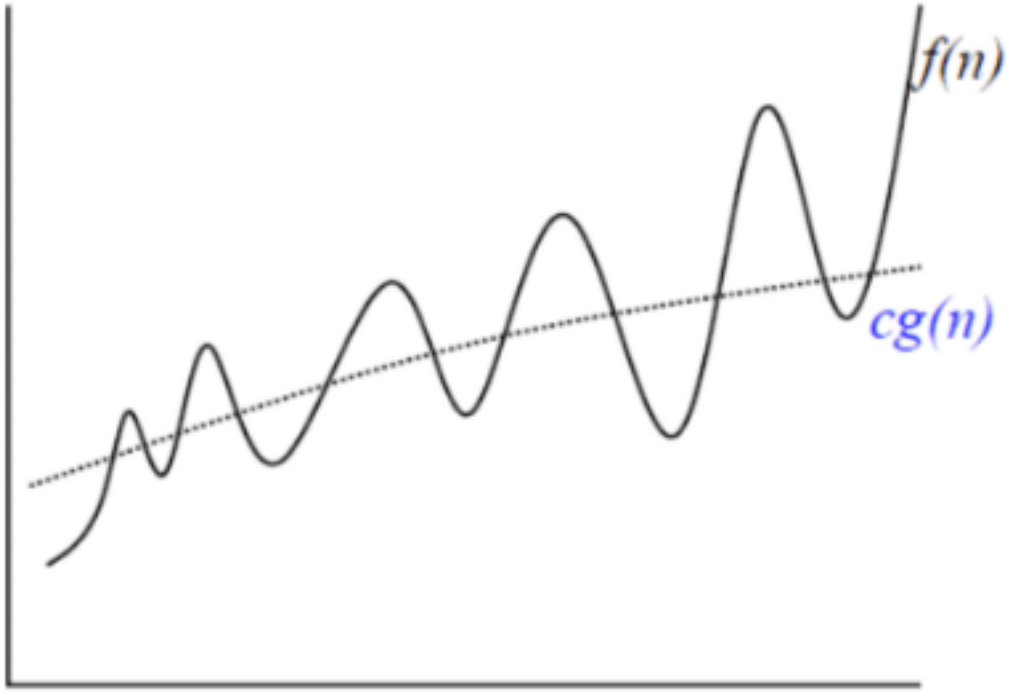
```
function multiply(y, z)
{
    x = 0;
    while z > 0 do
        if z is odd then x = x + y;
        y = 2y; z = [z/2];
    return x;
}
```

Nội dung

1. Quy nạp toán học
2. Tính đúng đắn của thuật toán
3. Thuật toán đệ quy
4. Thuật toán không đệ quy
5. Big Oh

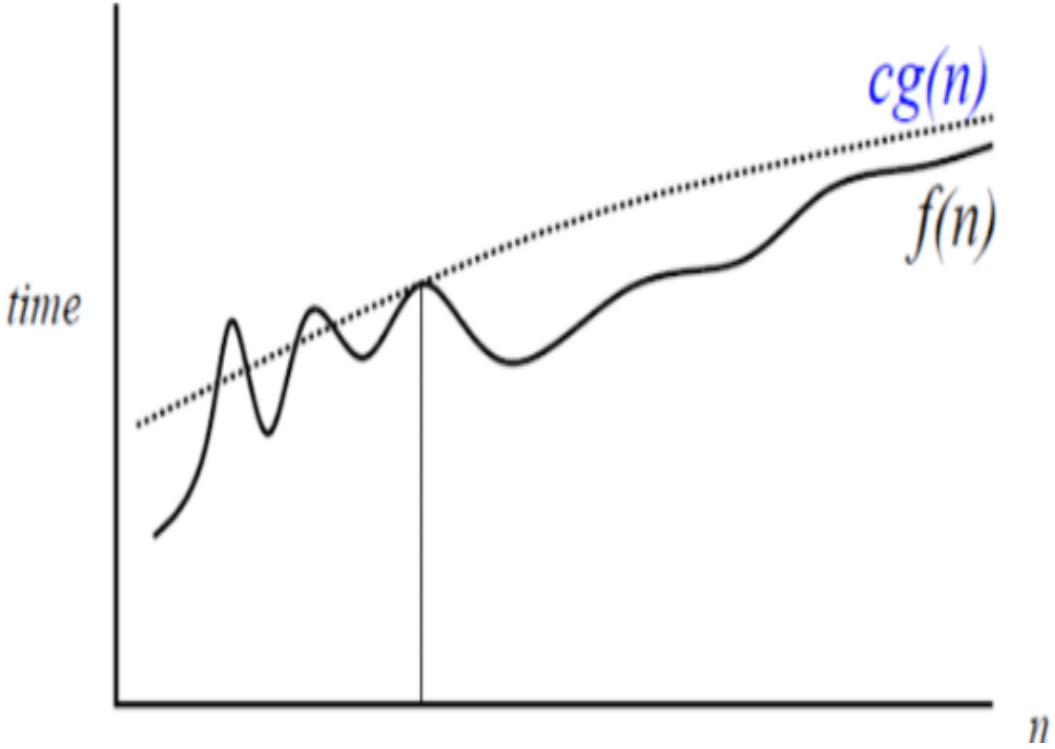
Big Oh

Ta nói: $f(n) = O(g(n))$ nếu tồn tại $c > 0$ sao cho với mọi $n > N_0$ thỏa: $|f(n)| < c|g(n)|$.



Big Oh

Chứng minh: $log(n) = O(n)$
Chứng minh: $2^{(n+1)} = O(3^n/n)$



Tính chất Big Oh

Một số tính chất của Big Oh:

- Nếu $f_1(n) = O(g_1(n))$ và $f_2(n) = O(g_2(n))$ thì

$$f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$$

- Nếu $f_1(n) = O(g_1(n))$ và $f_2(n) = O(g_2(n))$ thì

$$f_1(n) + f_2(n) = O(max(g_1(n), g_2(n)))$$

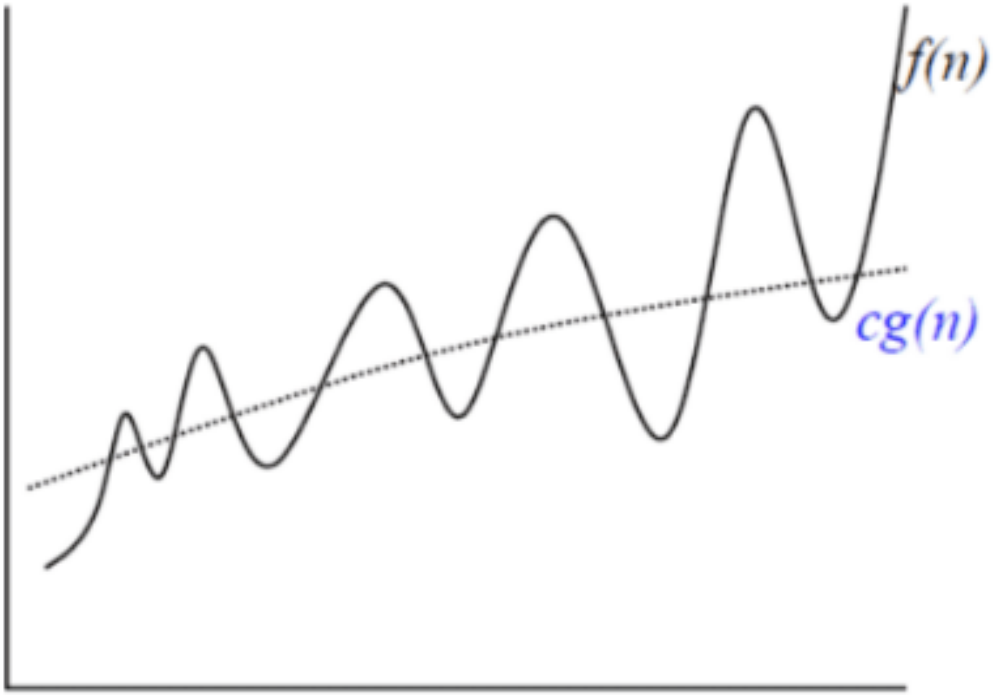
- Nếu $f_1(n) = O(g_1(n))$ và $f_2(n) = O(g_2(n))$ thì

$$f_1(n) * f_2(n) = O(g_1(n) * g_2(n))$$

Chứng minh.

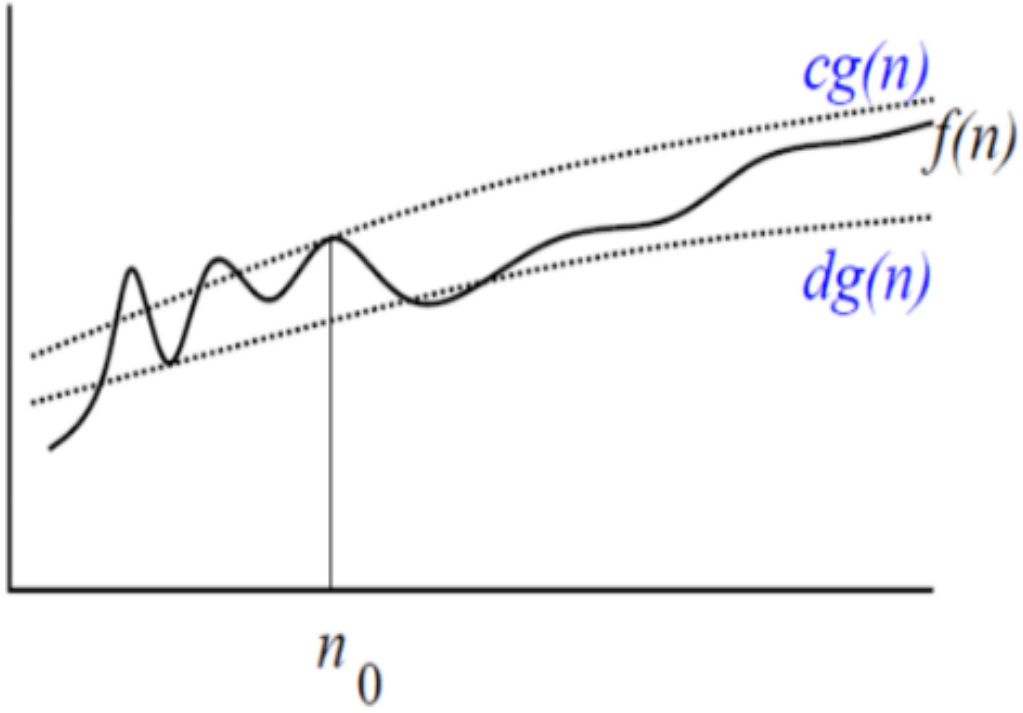
Big Omega

Ta nói: $f(n) = \Omega(g(n))$ nếu tồn tại $c > 0$ sao cho có vô hạn số n thỏa: $|f(n)| > c|g(n)|$.



Big Theta

Ta nói: $f(n) = \Theta(g(n))$ nếu và chỉ nếu $f(n) = O(g(n))$ và $g(n) = O(f(n))$



Ví dụ

Cho $f(n) = 4n^6 - 7n + 3$
Đúng hay sai?

- $f(n) = O(n^7)$?
- $f(n) = O(n^3)$?
- $f(n) = O(n^6)$?
- $f(n) = \Omega(n^7)$?
- $f(n) = \Omega(n^3)$?
- $f(n) = \Omega(n^6)$?
- $f(n) = \Theta(n^7)$?
- $f(n) = \Theta(n^3)$?
- $f(n) = \Theta(n^6)$?

Type of Analysis

- Worst case: độ phức tạp $T(n)$ trong trường hợp xấu nhất có thể. $T(n)$ đạt giá trị maximum trên mọi input có kích thước n.
- Average case: độ phức tạp trung bình dựa trên một phân phối xác suất của input.
- Probabilistic: độ phức tạp trung bình đối với input lấy giá trị ngẫu nhiên.

Type of Analysis

Xét một thuật toán mà tất cả các input n bit mất $2n$ thời gian và có một input có kích thước n có độ phức tạp n^n.

- Worst case: $O(n^n)$.
- Average case: $O(\frac{n^n+(2^n-1)2n}{2^n})$
- Probabilistic: ta nói thuật toán có độ phức tạp $O(n)$ với xác suất $1-1/2^n$.

Type of Analysis

Đánh giá độ phức tạp thuật toán sau:

```
s = 0
i = 1
while i ≤ n do
    j = n - i
    while j ≥ 1 do
        s = s + 1
        j = j - 1
    endw
    i = i +1
endw
```

Type of Analysis

Đánh giá độ phức tạp thuật toán sau trong trường hợp xấu nhất?

```
procedure bubblesort(A[1...n])
    for i := 1 to n - 1 do
        for j := 1 to n - i do
            if A[j] > A[j + 1] then
                Swap A[j] with A[j + 1]
```

Type of Analysis

Đánh giá độ phức tạp thuật toán sau trong trường hợp xấu nhất?

```
function maximum(A, n)
{
    m = A(1); i = 2;
    while i ≤ n do
        if A[i] > m then m = A(i);
    i = i + 1
    return m
}
```

Thank you for listening!